

Backend ↔ Frontend Architecture (FastAPI + Next.js Dashboard)
Technical Explanation for Full-Stack API Engineers
Created by:

Piezo-Pico to Femtotechnology Sensors Inc.

1. Overview

This document explains how the frontend (Next.js dashboard) communicates with the backend (FastAPI), how requests flow through main.py, how routers are mounted, and how pages like app/dashboard/page.tsx load data from the API. It includes a full architecture diagram showing the request lifecycle from the browser to the API and back.

2. Backend Architecture (FastAPI)

2.1 main.py — Application Entry Point

main.py is the root of your backend. It performs three critical tasks:

1. Creates the FastAPI application instance
2. Loads and mounts routers
3. Configures middleware (CORS, authentication, logging)

Example structure:

```
python
```

```
app = FastAPI()
```

```
app.include_router(auth_router, prefix="/auth")
```

```
app.include_router(cart_router, prefix="/cart")
```

```
app.include_router(payment_router, prefix="/payment")
```

```
app.include_router(docs_router, prefix="/docs")
```

This means all frontend requests must pass through main.py first, then get routed to the correct module.

3. Routers (Modular API Design)

Routers are independent API modules that handle specific domains:

Router	Purpose
<code>auth_router.py</code>	Login, register, tokens
<code>cart_router.py</code>	Add to cart, remove, list items
<code>payment_router.py</code>	Stripe checkout, webhook verification
<code>docs_router.py</code>	Serve documentation files
<code>user_router.py</code>	Profile, settings

Each router contains:

- Endpoints
- Request/response models
- Business logic
- Database interactions

Routers keep your backend clean, scalable, and easy to maintain.

4. Frontend Architecture (Next.js)

4.1 `app/dashboard/page.tsx`

This is the main dashboard page. It performs:

- Fetching data from backend APIs
- Rendering UI components
- Handling user interactions
- Displaying documentation, payments, courses, etc.

Example:

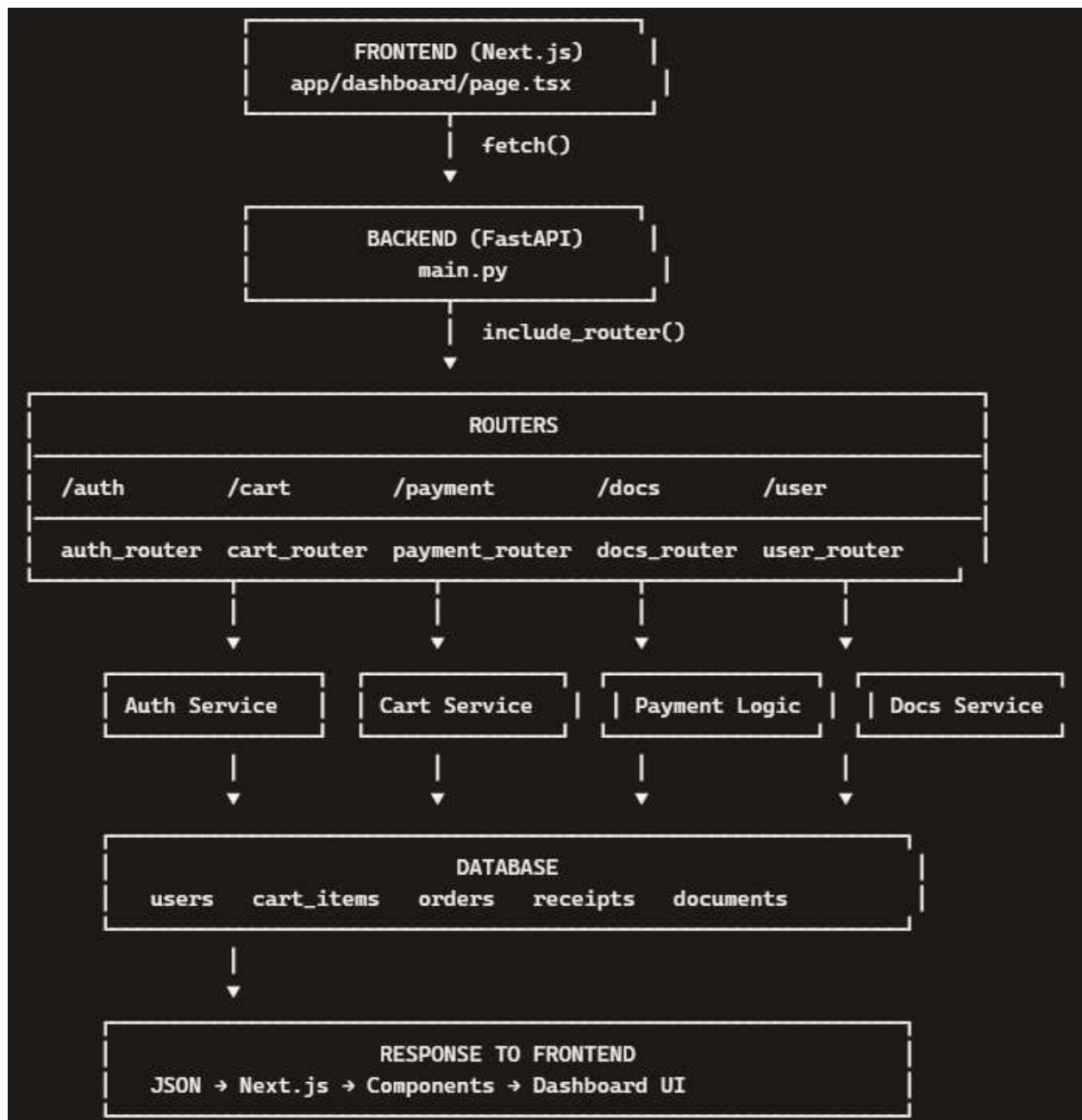
```
const response = await fetch("https://yourapi.com/docs/list");
const docs = await response.json();
```

The dashboard never talks directly to the database — it only communicates with FastAPI.

5. Full Request Lifecycle (Frontend → Backend → Routers → DB → Frontend)

Below is the complete system diagram showing how everything interacts.

6. System Architecture Diagram



7. How Frontend Loads Documentation

1. Dashboard calls: GET /docs/list
2. Backend returns list of documents from DB
3. Dashboard displays them in a sidebar
4. User clicks a document
5. Dashboard calls: GET /docs/{doc_id}
6. Backend returns PDF or Markdown
7. Dashboard renders the document viewer

8. How Frontend Loads Payment Data

1. Dashboard calls: GET /cart/items
2. User clicks “Checkout”
3. Dashboard calls: POST /payment/create-session
4. Backend returns Stripe checkout URL
5. Dashboard redirects user
6. Stripe processes payment
7. Stripe sends webhook to backend
8. Backend updates database
9. Dashboard fetches updated status

9. Publication & Contact Information

Publisher:

Piezo Pico-to-Femtotechnology Sensors Inc. Edmonton, Alberta, Canada

Contact:

- **Email:** selemaniseif12@yahoo.com or selemaniseif1974@gmail.com
- **Website:** Piezo Pico to Femtotechnology Sensors Inc.
- **Support:** selemaniseif12@yahoo.com
- **Phone:** 780 6044032

Copyright

© 2026 Piezo Pico-to-Femtotechnology Sensors Inc. All rights reserved.

9. Summary

This document provides a complete architectural overview of how your frontend interacts with your backend through `main.py`, routers, and the dashboard page. It is suitable for your documentation system, your engineering course, and your commercial API platform.